AddOperationModal

Plan de arquitectura y prompt de implementación

Stack	React + TypeScript + Tailwind CSS
Patrón	Feature-based, componentes modulares, hook centralizado
Ficheros	18 ficheros (1 modal, 1 hook, 1 types, 15 componentes)
Steps	3 pasos (Operación → Partes → Resumen)

1. Estructura de ficheros

AddOperationModal.tsx ← orquestador principal

useOperationForm.ts ← estado, validación, navegación

operation.types.ts ← interfaces y enums

StepIndicator.tsx ← barra de progreso

ModalHeader.tsx ← título + botón X

ModalFooter.tsx ← volver + continuar/guardar

Step1Operation.tsx ← tipo, moneda, campos

Step2Parties.tsx ← deudor o acreedor

Step3Summary.tsx ← resumen + guardar

OperationTypeToggle.tsx ← compra / venta

CurrencySelect.tsx ← divisas y metales

MetalFields.tsx ← quilataje + peso (opcionales)

PartyToggleGroup.tsx ← deudor / acreedor toggles

DebtorFields.tsx ← nombre + importe

CreditorFields.tsx ← nombre, moneda, fecha

FormField.tsx ← label + input + error

ToggleSwitch.tsx ← toggle accesible

SectionDivider.tsx ← separador visual

ValidationMessage.tsx ← error en rojo

2. Tipos — operation.types.ts

Fuente única de verdad para todo el tipado. Ningún componente redefine tipos propios.

```
export type OperationType = 'Compra' | 'Venta'
export type StepId = 1 | 2 | 3
export const METAL_CURRENCIES = ['Oro', 'Plata', 'Platino', 'Paladio'] as const
export const FIAT_CURRENCIES  = ['EUR', 'USD', 'GBP', 'CHF', 'JPY']  as const
export type Currency = typeof METAL_CURRENCIES[number] | typeof FIAT_CURRENCIES[number] | ''

export interface OperationForm {
  opType: OperationType
  currency: Currency
  karat: string
  weight: string
  quantity: string
  quote: string
  hasDebtor: boolean
  hasCreditor: boolean
  debtorName: string
  debtorAmount: string
  creditorName: string
  creditorAmount: string
  creditorDueDate: string
}

export interface StepValidation {
  isValid: boolean
  errors: Partial<Record<keyof OperationForm, string>>
}
```

3. Hook — useOperationForm.ts

Centraliza todo el estado y la lógica. Ningún componente tiene useState propio para datos del formulario.

Retorna:

```
{
  form,               // OperationForm completo
  updateField,        // <K extends keyof OperationForm>(key: K, value: OperationForm[K]) => void
  step,               // StepId actual
  goNext,             // avanza; si step 1 y !needsStep2 salta directamente a 3
  goBack,             // retrocede; si step 3 y !needsStep2 vuelve a 1
  needsStep2,         // boolean: hasDebtor || hasCreditor
  isMetal,            // boolean: METAL_CURRENCIES incluye currency
  computedAmount,     // string: (parseFloat(quantity) * parseFloat(quote)).toFixed(2) o ''
  isCurrentStepValid, // boolean
  errors,             // Partial<Record<keyof OperationForm, string>>
  showErrors,         // boolean
  triggerValidation,  // () => void — activa showErrors y recalcula errors
  resetShowErrors,    // () => void
}
```

Reglas de validación:

Step 1 — obligatorios	currency, quantity (> 0), quote (> 0)
Step 1 — opcionales	karat y weight (solo disponibles si isMetal)

Step 2 — deudor	debtorName (no vacío), debtorAmount (> 0)
Step 2 — acreedor	creditorName (no vacío), creditorAmount (> 0), creditorDueDate (no vacío)
Efecto isMetal → false	Limpiar karat y weight automáticamente con useEffect
Efecto hasDebtor	Al activarse forzar hasCreditor = false y viceversa
computedAmount	Se usa como valor por defecto de debtorAmount / creditorAmount al entrar al step 2

4. Componentes compartidos

StepIndicator.tsx

Props: `currentStep: StepId, skippedSteps: StepId[]`

Renderiza siempre los 3 pasos. Un step está completado si currentStep > stepId o si skippedSteps lo incluye y currentStep === 3. Completado → círculo verde con ✓. Activo → círculo verde. Pendiente → gris. Línea conectora verde si el step izquierdo está completado.

ModalHeader.tsx

Props: `title: string, onClose: () => void`

Título a la izquierda, botón X a la derecha. Sin lógica de formulario.

ModalFooter.tsx

Props: `onBack, onNext, isBackDisabled, isNextDisabled, nextLabel: "Continuar" | "Guardar"`

Botón "← Volver" deshabilitado visualmente cuando isBackDisabled. Botón derecho gris + cursor not-allowed cuando isNextDisabled, verde y activo cuando no.

5. Steps

Step1Operation.tsx

Props: form, updateField, errors, showErrors, isMetal

Compone en orden:

1. OperationTypeToggle
2. CurrencySelect
3. MetalFields — solo renderizar si isMetal
4. FormField para quantity (number, obligatorio)
5. FormField para quote (number, obligatorio)
6. SectionDivider
7. PartyToggleGroup

Step2Parties.tsx

Props: form, updateField, errors, showErrors, computedAmount

Si form.hasDebtor → renderiza DebtorFields. Si form.hasCreditor → renderiza CreditorFields. Si ninguno → párrafo informativo (no debería ocurrir, pero cubrirlo).

Step3Summary.tsx

Props: form, isMetal, computedAmount

Tipo, Moneda	Siempre visibles
Quilataje, Peso	Solo si isMetal y tienen valor
Cantidad, Cotización, Importe	Siempre visibles
Deudor (nombre)	Solo si hasDebtor
Acreedor (nombre), Moneda acreedor, Fecha	Solo si hasCreditor

6. Subcomponentes de Step 1

OperationTypeToggle.tsx

Props: `value: OperationType, onChange: (v: OperationType) => void`

Dos botones tipo toggle: "Compra" / "Venta". Activo en verde (green-600), inactivo en gris.

CurrencySelect.tsx

Props: `value: Currency, onChange: (v: Currency) => void, error?: string, showError?: boolean`

Select con dos : "Divisas" (FIAT_CURRENCIES) y "Metales" (METAL_CURRENCIES). Usa FormField internamente con el label "Moneda".

MetalFields.tsx

Props: `karat: string, weight: string, onChange: (field: 'karat' | 'weight', value: string) => void`

Select de quilataje: opciones vacío, 9k, 14k, 18k, 24k. Label "Quilataje (opcional)". Input number para peso. Label "Peso en gramos (opcional)". Ambos usan FormField.

PartyToggleGroup.tsx

Props: `hasDebtor: boolean, hasCreditor: boolean, onToggleDebtor, onToggleCreditor`

Dos filas con ToggleSwitch. El toggle de deudor se deshabilita si hasCreditor === true y viceversa. Al activar uno llama a updateField para desactivar el otro.

7. Subcomponentes de Step 2

DebtorFields.tsx

Props: `name, amount, onChange: (field: 'debtorName' | 'debtorAmount', value: string) => void, errors, showErrors`

FormField para nombre (text, obligatorio). FormField para importe (number, obligatorio, valor inicial = computedAmount si vacío).

CreditorFields.tsx

Props: `name, amount, dueDate, currency: Currency, onChange: (field: 'creditorName' | 'creditorAmount' | 'creditorDueDate', value: string) => void, errors, showErrors`

FormField para nombre (obligatorio). FormField para moneda (input readonly con el valor de currency). FormField para importe (number, obligatorio). FormField para fecha de vencimiento (input type date, obligatorio).

8. UI atómico reutilizable

Estos componentes no importan nada de operation.types.ts. Son agnósticos al dominio.

FormField.tsx

Props: `label: string, children: React.ReactNode, error?: string, showError?: boolean`

Renderiza label arriba, children (el input/select), y si showError && error renderiza ValidationMessage.

ToggleSwitch.tsx

Props: `checked: boolean, onChange: (v: boolean) => void, disabled?: boolean, label: string, description?: string`

Fila con info a la izquierda (label en negrita + description en gris) y el toggle a la derecha. Toggle CSS puro: input checkbox oculto + span con transición. Verde cuando activo, opacidad reducida si disabled.

SectionDivider.tsx

Props: `Sin props`

Línea horizontal separadora de sección (hr con color gris suave).

ValidationMessage.tsx

Props: `message: string`

Texto pequeño en rojo (text-red-600) debajo de un campo.

9. AddOperationModal.tsx — orquestador

Props: isOpen: boolean, onClose: () => void, onSave: (form: OperationForm) => void

1. Llama a useOperationForm()
2. Si !isOpen no renderiza nada (o usa portal)
3. Renderiza overlay oscuro + card centrada
4. Compone: ModalHeader, StepIndicator, el step activo, ModalFooter
5. goNext en step 3 llama a onSave(form) y cierra
6. skippedSteps para StepIndicator: !needsStep2 && step === 3 ? [2] : []

Mapeo de step a componente:

```
const stepComponents: Record<StepId, React.ReactNode> = {
  1: <Step1Operation form={form} updateField={updateField} errors={errors}
       showErrors={showErrors} isMetal={isMetal} />,
  2: <Step2Parties form={form} updateField={updateField} errors={errors}
```

```
        showErrors={showErrors} computedAmount={computedAmount} />,
  3: <Step3Summary form={form} isMetal={isMetal} computedAmount={computedAmount} />,
}

return (
  <div className="fixed inset-0 bg-black/40 flex items-center justify-center z-50">
    <div className="bg-white rounded-xl w-full max-w-md p-7 shadow-lg">
      <ModalHeader title="Nueva operación" onClose={onClose} />
      <StepIndicator currentStep={step} skippedSteps={!needsStep2 && step === 3 ? [2] : []} />
      {stepComponents[step]}
      <ModalFooter
        onBack={goBack}
        onNext={step === 3 ? () => { onSave(form); onClose(); } : goNext}
        isBackDisabled={step === 1}
        isNextDisabled={!isCurrentStepValid}
        nextLabel={step === 3 ? 'Guardar' : 'Continuar'}
      />
    </div>
  </div>
)
```

10. Restricciones y buenas prácticas

Estilos	Tailwind CSS exclusivamente. Color primario/acción: green-600 (#16a34a). Sin estilos inline salvo casos puntu
Tipado	No usar any. Tipado estricto en todo momento.
Lógica	Ningún componente de UI tiene lógica de negocio. Toda está en useOperationForm.
Constantes	METAL_CURRENCIES y FIAT_CURRENCIES importados siempre desde operation.types.ts, nunca redefinido
Exports	Cada fichero exporta un único componente o hook como export default.
Completitud	Generar todos los ficheros completos, sin placeholders ni // TODO.
UI atómico	FormField, ToggleSwitch, etc. no importan nada de operation.types.ts.
computedAmount	Se calcula en el hook y se pasa como prop, no se recalcula en cada render.
MetalFields	Se limpia con useEffect para no dejar datos huérfanos al cambiar moneda.
Modularidad	Un fichero = un componente o hook. Sin componentes anidados en el mismo fichero.

11. Prompt completo para Claude

Copia el texto a continuación y pégalo directamente en Claude para generar todo el código.

Genera el código completo de un modal multi-step en React + TypeScript llamado **AddOperationModal.tsx** siguiendo estrictamente la arquitectura modular descrita en este documento. Crea todos los ficheros indicados en la sección de estructura, sin omitir ninguno, aplicando todas las restricciones de la sección 10.

A continuación adjunta este documento completo (o pega el contenido de las secciones 2–10) como contexto para que Claude tenga toda la especificación.